



THIRD EDITION

آموزش یونیتی

Unity in action



Unity
IN ACTION



Unity IN ACTION

THIRD EDITION

Multiplatform game development in C#



Joseph Hocking

Foreword by Jesse Schell



Adding enemies and projectiles to the 3D game

اضافه کردن دشمن و پرتابه ها به پروژه
بازی سه بعدی

This chapter covers

- Taking aim and firing, both for the player and for enemies
- Detecting and responding to hits
- Making enemies that wander around
- Spawning new objects in the scene



پروژه حرکت از فصل قبل بسیار جالب بود اما هنوز واقعاً یک بازی نیست. بیا ببینیم آن دمو حرکت را به یک تیراندازی اول شخص تبدیل کنیم. اگر به چیزهای دیگری که اکنون نیاز داریم فکر کنید، توانایی تیراندازی و داشتن وسایلی برای شلیک کردن خلاصه می شود.

ابتدا، ما اسکریپت هایی را می نویسیم که بازیکن را قادر می سازد به اشیاء در صحنه شلیک کند. سپس، ما دشمنانی را برای پر کردن صحنه می سازیم، از جمله کدهایی برای پرسه زدن بی هدف و واکنش به ضربه خوردن. در نهایت، ما دشمنان را قادر می سازیم تا با شلیک گلوله های آتشین به سمت بازیکن، مقابله کنند. هیچ یک از اسکریپت های فصل ۲ نیازی به تغییر ندارند. در عوض، ما اسکریپت هایی را به پروژه اضافه می کنیم - اسکریپت هایی که ویژگی های اضافی را مدیریت می کنند.

نقشه راه این فصل

۱. کدی را بنویسید که بازیکن را قادر می سازد به صحنه شلیک کند.
۲. اهداف ایستا ایجاد کنید که به ضربه زدن واکنش نشان می دهند.
۳. اهداف را در اطراف سرگردان کنید.
۴. اهداف سرگردان را به طور خودکار ایجاد و پراکنده شوند.
۵. اهداف/دشمنان را فعال کنید تا به سمت بازیکن شلیک کنند.



Shooting via raycasts

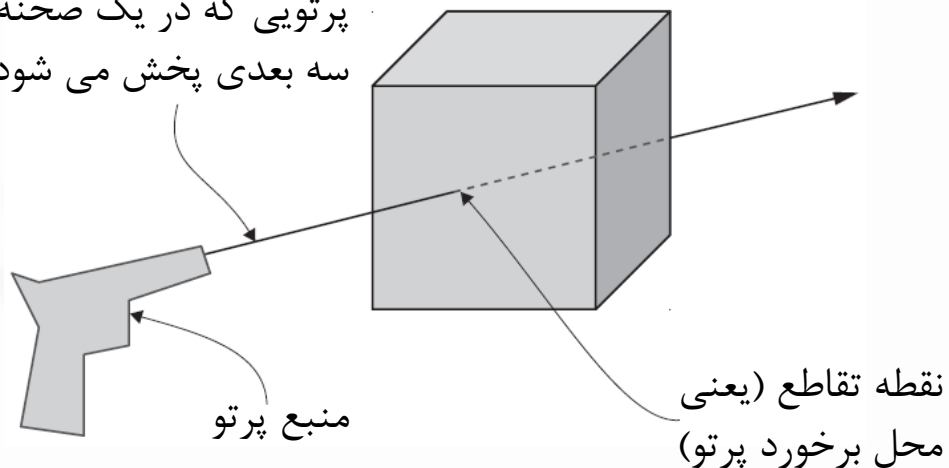
تیراندازی در بازی های سه بعدی با چند رویکرد قابل پیاده سازی است و یکی از مهم ترین رویکردها، raycasting است.

DEFINITION A ray is an imaginary or invisible line in the scene that starts at a point of origin and extends out in a specific direction.

پرتو یک خط خیالی یا نامرئی در صحنه است که از یک نقطه مبدأ شروع می شود و در یک جهت خاص امتداد می یابد.

- در raycasting، شما یک پرتو ایجاد می کنید و سپس تعیین می کنید که چه چیزی آن را قطع می کند. شکل زیر این مفهوم را نشان می دهد.
- در نظر بگیرید که وقتی گلوله ای را از تفنگ شلیک می کنید چه اتفاقی می افتد: گلوله از موقعیت تفنگ شروع می شود و سپس در یک خط مستقیم به جلو حرکت می کند تا زمانی که به چیزی برخورد کند. پرتو مشابه مسیر گلوله است و پرتو افشانی شبیه شلیک گلوله و دیدن چیزی است که به آن اصابت می کند.

پرتویی که در یک صحنه
سه بعدی پخش می شود



برای بازی اول شخص، پرتو به طور کلی از موقعیت دوربین شروع می شود و سپس از مرکز نمای دوربین به بیرون امتداد می یابد. به عبارت دیگر، شما در حال بررسی اشیاء مستقیم در مقابل دوربین هستید. یونیتی دستوراتی را برای ساده کردن این کار ارائه می دهد. بیایید به این دستورات نگاه کنیم.



Using the ScreenPointToRay command for shooting

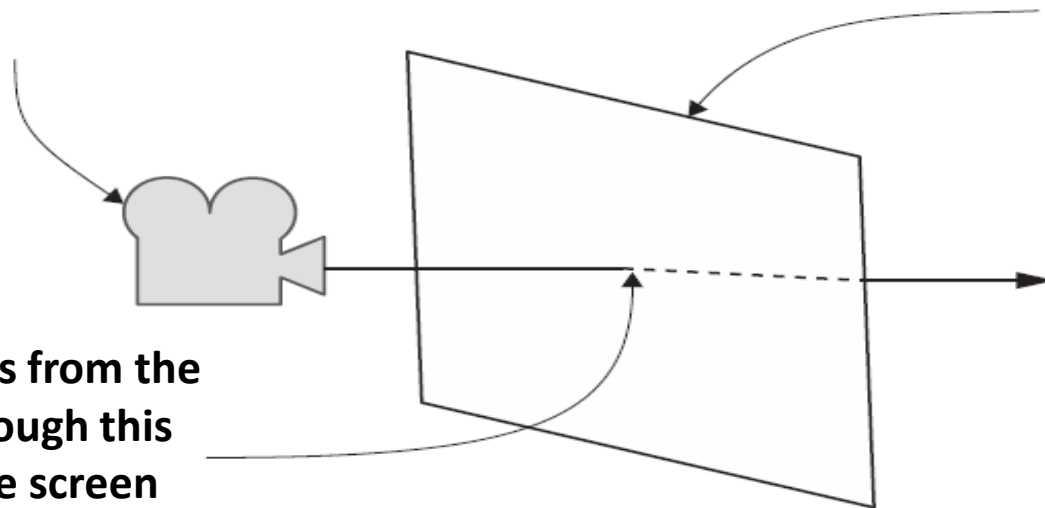
- تیراندازی با استفاده از تاباندن یک پرتو که از دوربین شروع می شود پیاده سازی خواهد شد. Unity تابع ScreenPointToRay() را برای انجام این عمل فراهم می کند.

Shooting via raycasts

The camera is the origin of this ray, similar to the gun previously.

The screen (i.e., the camera's window into the 3D scene)

Ray projects from the camera through this point on the screen



- اگر می خواهید یک پرتو داشته باشید می توانید از تابع Physics.Raycast() استفاده کنید



- بیایید کدی بنویسیم که از روش هایی که قبلاً بحث کردیم استفاده کند. در Unity، یک اسکریپت C# جدید ایجاد کنید، آن را RayShooter نامید، آن اسکریپت را به دوربین (نه شی پخش کننده) وصل کنید

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class RayShooter : MonoBehaviour {
    private Camera cam;
    void Start() {
        cam = GetComponent<Camera>();
    }
    void Update() {
        if (Input.GetMouseButtonDown(0)) {
            Vector3 point = new Vector3(cam.pixelWidth/2, cam.pixelHeight/2, 0);
            Ray ray = cam.ScreenPointToRay(point);
            RaycastHit hit;
            if (Physics.Raycast(ray, out hit)) {
                Debug.Log("Hit " + hit.point);
            }
        }
    }
}
```

**Access other components
attached to the same object.**

**Respond to the left (first)
mouse button.**

**Create the ray at that
position by using
ScreenPointToRay().**

**The raycast fills a
referenced variable
with information.**

**Retrieve coordinates
where the ray hit.**

**The
middle
of the
screen is
half its
width
and
height.**



- متد `Input.GetMouseButtonDown()` بسته به اینکه روی ماوس کلیک شده باشد، `true` یا `false` را برمی گرداند، بنابراین قرار دادن این دستور در یک شرط به این معنی است که کد محصور شده تنها زمانی اجرا می شود که روی ماوس کلیک شده باشد. وقتی بازیکن روی ماوس کلیک می کند می خواهید شلیک کنید—بنابراین دکمه ماوس را به صورت مشروط بررسی کنید.

- یک بردار برای تعریف مختصات صفحه برای پرتو ایجاد می شود

- مقادیر پیکسل `pixelWidth` و `pixelHeight` اندازه صفحه را به شما می دهد، بنابراین تقسیم آن مقادیر به دو، مرکز صفحه را به شما می دهد.

- پرتو `ray` به متد `Raycast()` ارسال می شود، اما این تنها شیء ارسالی نیست. ساختار داده `RaycastHit` نیز وجود دارد. `RaycastHit` مجموعه ای از اطلاعات در مورد برخورد پرتو است، از جمله محل تقاطع و اینکه چه شیئی قطع شده است. دستور `out` تضمین می کند که ساختار داده ای یا متغیری که به تابع ارسال می شود، همان شیئی است که خارج از تابع وجود دارد.

- `Physics.Raycast()` : این روش تقاطع ها را با پرتو داده شده بررسی می کند، داده های مربوط به تقاطع را پر می کند و اگر پرتو به چیزی برخورد کند `true` را برمی گرداند.

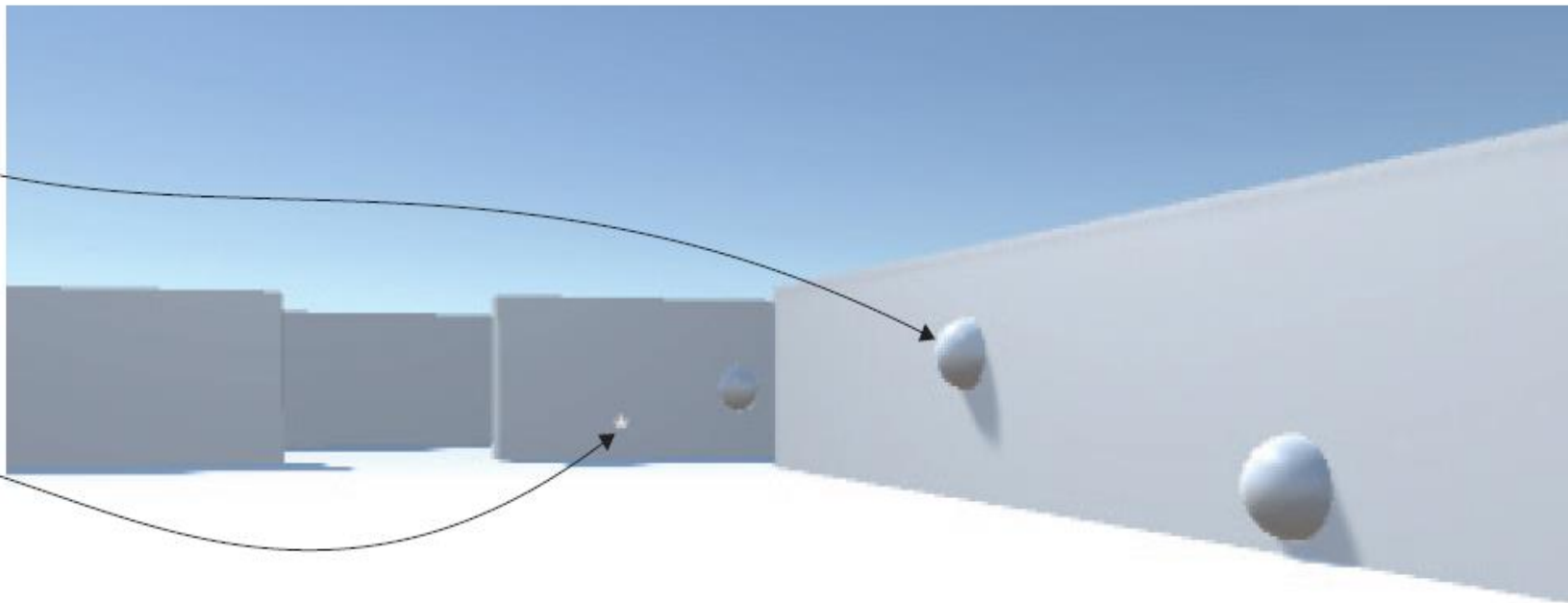




Adding visual indicators for aiming and hits

**Sphere indicates
where the wall
was hit**

**Target point in the
center of the screen**



می‌خواهید یک کره خالی در محل برخورد تیر با دیوار قرار دهید (و بعد از ۱ ثانیه از یک کوروتین برای برداشتن و حذف کره استفاده کنید).



Coroutines



- به طور کلی ، اگر یک تابع را در Unity (یا C#) فراخوانی کنید ، عملکرد از ابتدا تا انتها اجرا خواهد شد. این رفتار عادی در یک کد است که می توان در نظر گرفت. با این حال، بعضی اوقات می خواهیم به طور عمدی یک عملکرد را انجام ندهیم یا آن را بعد از چند ثانیه اجرا کنیم.
- یک کوروتین به شما امکان می دهد وظایف را در چندین فریم پخش کنید. در یونیتی، کوروتین روشی است که می تواند اجرا را متوقف کند و کنترل را به یونیتی برگرداند، اما سپس در فریم های بعدی از همان جایی که متوقف شد ادامه دهد.
- در بیشتر موقعیت ها، وقتی متدی را فراخوانی می کنید، تا پایان اجرا می شود و سپس کنترل را به متد فراخوانی به علاوه هر مقدار بازگشتی اختیاری برمی گرداند. این بدان معنی است که هر عملی که در یک متد انجام می شود باید در یک به `update()` یک فریم اتفاق بیفتد.
- در مواقعی که می خواهید از فراخوانی متد برای حاوی یک انیمیشن رویه ای یا توالی رویدادها در طول زمان استفاده کنید، می توانید از کوروتین استفاده کنید.
- با این حال، مهم است که به یاد داشته باشید که کوروتین ها رشته نیستند. عملیات همزمانی که در یک کوروتین اجرا می شوند همچنان روی رشته اصلی اجرا می شوند. اگر می خواهید مقدار زمان صرف شده در CPU روی رشته اصلی را کاهش دهید، به همان اندازه مهم است که از مسدود کردن عملیات در کوروتین ها مانند سایر کدهای اسکرپت خودداری کنید.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class RayShooter : MonoBehaviour {
    private Camera cam;

    void Start() {
        cam = GetComponent<Camera>();
    }

    void Update() {
        if (Input.GetMouseButtonDown(0)) {
            Vector3 point = new Vector3(cam.pixelWidth/2, cam.pixelHeight/2, 0);
            Ray ray = cam.ScreenPointToRay(point);
            RaycastHit hit;
            if (Physics.Raycast(ray, out hit)) {
                StartCoroutine(SphereIndicator(hit.point));
            }
        }
    }

    private IEnumerator SphereIndicator(Vector3 pos) {
        GameObject sphere = GameObject.CreatePrimitive(PrimitiveType.Sphere);
        sphere.transform.position = pos;
        yield return new WaitForSeconds(1);
        Destroy(sphere);
    }
}
```

- متد جدید `SphereIndicator()`، به اضافه یک اصلاح یک خطی در متد `Update()` موجود است. این روش یک کره را در نقطه ای از صحنه ایجاد می کند و یک ثانیه بعد آن کره را حذف می کند. فراخوانی `SphereIndicator()` از کد `raycasting` تضمین می کند که نشانگرهای بصری وجود دارد که دقیقاً محل برخورد پرتو را نشان می دهد. این تابع با `IEnumerator` تعریف شده



- از نظر فنی، کوروتین‌ها ناهمزمان (آسنکرون asynchronous) نیستند (عملیات ناهمزمان اجرای بقیه کد را متوقف نمی‌کند؛ به داندلود یک تصویر در اسکریپت یک وب سایت فکر کنید)، اما با استفاده هوشمندانه از شمارشگرها (enumerators)، Unity باعث می‌شود که کوروتین‌ها مشابه عملکردهای ناهمزمان رفتار کنند..
- روش کار به این صورت است که کوروتین به طور موقت متوقف شود (در حقیقت تابعی که آن را شروع کردیم برای کوروتین متوقف می‌شود)، اما کنترل به تابع فراخواننده باز می‌گردد و سپس دوباره از آن نقطه در فریم بعدی می‌شود. به این ترتیب، کوروتین‌ها ظاهراً در پس‌زمینه برنامه اجرا می‌شوند، از طریق یک چرخه مکرر اجرای نیمه‌راهه و سپس بازگشت به بقیه برنامه.
- بر خلاف سایر زبان برنامه نویسی برای اجرای کوروتین‌ها در یونیتی thread جدیدی ایجاد نمی‌شود





- ما می‌توانیم نشانگر موس را مخفی کرده و همچنین یک علامت در وسط صفحه نمایش برای نشانه گرفتن اضافه کنیم
- برای مخفی کردن علامت پیش فرض موس تابع `start` را بصورت زیر ویرایش می‌کنیم
- برای رسم نشانگر خودمان در وسط صفحه از تابع `OnGUI` استفاده خواهیم کرد.
- این تابع مشابه تابع های `Start` و `Update` است و مانند آنها بطور اتوماتیک در هر فریم صدا زده میشود بعد از آن که تمام اشیاء سه بعدی رندر `Render` شدن و رسم آنها کامل شد این تابع فراخوانی می‌شود و به همین دلیل تصاویر ساخته شده در این تابع بالاتر از جسم های سه بعدی قرار می‌گیرد

...

```
void Start() {  
    cam = GetComponent<Camera>();  
    Cursor.lockState = CursorLockMode.Locked;           نشانگر ماوس را در مرکز  
    Cursor.visible = false;                             صفحه پنهان کنید.  
}
```

```
void OnGUI() {  
    int size = 12;  
    float posX = cam.pixelWidth/2 - size/4;  
    float posY = cam.pixelHeight/2 - size/2;  
    GUI.Label(new Rect(posX, posY, size, size), "*") ←  
}
```

The `GUI.Label()` command displays text onscreen.

...

در داخل `OnGUI()`، کد مختصات دوبعدی را برای نمایشگر تعریف می‌کند (برای محاسبه اندازه برچسب کمی تغییر می‌کند) و سپس `GUI.Label()` را فراخوانی می‌کند. این روش یک برچسب متنی را نمایش می‌دهد. از آنجا که رشته ارسال شده به برچسب یک ستاره (*) است، در نهایت آن کاراکتر در مرکز صفحه نمایش داده می‌شود.



رندر عملی است که کامپیوتر پیکسل های صحنه سه بعدی را ترسیم می کند. اگرچه صحنه با استفاده از مختصات x ، y و z تعریف می شود، اما نمایشگر واقعی روی مانیتور شما یک شبکه دو بعدی از پیکسل های رنگی است. برای نمایش صحنه سه بعدی، کامپیوتر باید رنگ تمام پیکسل های شبکه دو بعدی را محاسبه کند. اجرای آن الگوریتم به عنوان رندر نامیده می شود.

همیشه به یاد داشته باشید که می توانید ESC را فشار دهید تا مکان نمای ماوس باز شود تا آن را از وسط نمای بازی دور کنید. در حالی که نشانگر ماوس قفل است، نمی توان روی دکمه Play کلیک کرد و بازی را متوقف کرد.





Scripting reactive targets

این که بازیکن بتواند تیراندازی کند خوب است، اما در حال حاضر بازیکنان چیزی که بتوانند به آن شلیک کنند ندارند. ما یک شی هدف ایجاد می کنیم و به آن اسکریپتی می دهیم که به ضربه زدن پاسخ دهد. یا بهتر است بگوییم، ما کمی کد تیراندازی را تغییر می دهیم تا در هنگام برخورد به هدف، این اتفاق به اطلاع خود هدف نیز برسد و سپس اسکریپت روی هدف پس از اطلاع واکنش نشان می دهد..

Determining what was hit

در گام اول می خواهیم تشخیص دهیم که آیا تیر با هدف برخورد کرده است و یا به جسم دیگری برخورد کرده است

در صورت برخورد تیر با هدف یک پیام ساده به ما نشان داده می شود در غیر اینصورت مشابه قبل در محل برخورد یک Sphere ایجاد می شود

...

```
if (Physics.Raycast(ray, out hit)) {  
    GameObject hitObject = hit.transform.gameObject;
```

```
    ReactiveTarget target = hitObject.GetComponent<ReactiveTarget>();
```

Retrieve the object
the ray hit.

```
    if (target != null) {  
        Debug.Log("Target hit");
```

```
    } else {
```

```
        StartCoroutine(SphereIndicator(hit.point));
```

```
    }
```

```
}
```

...

Check for the ReactiveTarget
component on the object.



اگر برخوردی رخ داده باشد ابتدا جسمی که برخورد به آن رخ داده است را بازیابی می کنیم
اگر به این جسم کامپوننت متصل باشد پس متوجه می شویم تیر به هدف برخورد کرده است

```
GameObject hitObject = hit.transform.gameObject;  
ReactiveTarget target =  
hitObject.GetComponent<ReactiveTarget>() ;  
if (target != null)
```





Alerting the target that it was hit

هشدار دادن به هدف که مورد اصابت قرار گرفته است

- تنها چیزی که در کد مورد نیاز است تغییر یک خطی است، همانطور که در ادامه نشان داده شده است.

```
...  
if (target != null) {  
    target.ReactToHit(); ←  
} else {  
    StartCoroutine(SphereIndicator(hit.point));  
}  
...
```

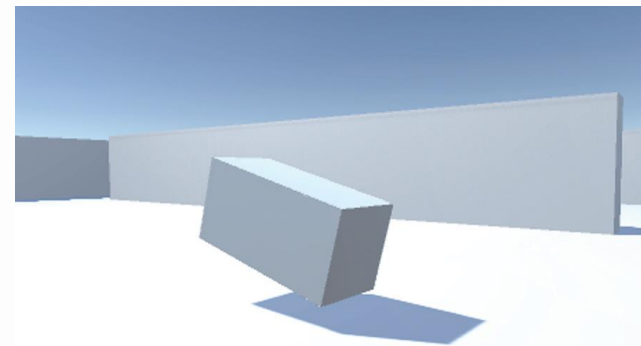
اگر تیر با هدف برخورد کرد باشد یک تابع از هدف مورد اصابت قرار گرفته را فراخوانی می کنیم تا عکس العمل مناسب را انجام دهد





- اکنون کد مربوط به عکس العمل مناسبی که باید توسط هدف زمانی که تیر با آن برخورد می کند ، را می نویسیم

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class ReactiveTarget : MonoBehaviour {
public void ReactToHit() {
    StartCoroutine(Die());
}
private IEnumerator Die() {
    this.transform.Rotate(-75, 0, 0);
    yield return new WaitForSeconds(1.5f);
    Destroy(this.gameObject);
}
}
```



تابعی که توسط تیر در هنگام اصابت فراخوانی میشود تا عکس العمل مناسب انجام شود

تابعی برای از بین رفتن هدفی که مورد اصابت تیر قرار گرفته است

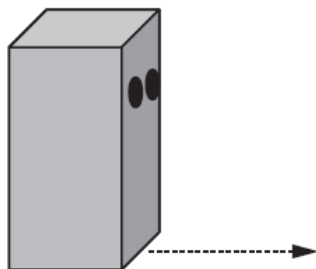
this.gameObject به شیئی اشاره دارد که اسکریپت به آن متصل شده است



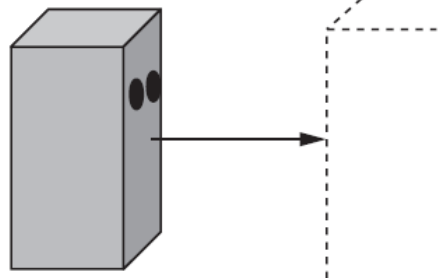
Basic wandering AI

- یک هدف ثابت چالب نیست، بنابراین بیایید کدی بنویسیم که دشمن بتواند در صحنه حرکت کند. کد برای سرگردانی تقریباً ساده‌ترین مثال از هوش مصنوعی است.

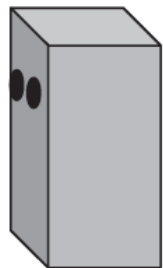
Step 1:
Move forward a little bit.



Step 2:
Raycast forward to look for obstacles.



Step 3:
Turn away from obstacles.



Step 4:
Frame rendered, return to step 1.

در هر فریم، کد هوش مصنوعی اطراف محیط خود را اسکن می کند تا مشخص کند که آیا نیاز به واکنش دارد یا خیر. اگر مانعی بر سر راه خود ظاهر شود، دشمن به سمت دیگری رو به رو می شود. صرف نظر از اینکه آیا دشمن نیاز به چرخش دارد، همیشه به طور پیوسته به جلو حرکت می کند. اگر در جلو خود دیواری را تشخیص دهد بصورت تصادفی جهت حرکت خود را تغییر می دهد



“Seeing” obstacles with a raycast

دیدن موانع با استفاده از تابش پرتو نامرئی

raycasting تکنیکی است که برای چندین کار در شبیه سازی های سه بعدی مفید است. یکی از کارهایی که به راحتی قابل درک است، تیراندازی است، اما کار دیگری که می تواند برای آن مفید باشد، اسکن صحنه است

قبلاً، یک پرتو ایجاد کردید که از دوربین شروع می شد، زیرا بازیکن از آنجا نگاه می کرد. این بار، پرتویی ایجاد می کنید که از دشمن سرچشمه می گیرد. پرتو اول از مرکز صفحه بیرون زد، اما این بار پرتو به جلو در مقابل شخصیت قرار می گیرد. سپس، همانطور که کد تیراندازی از اطلاعات RaycastHit برای تعیین اینکه آیا چیزی مورد اصابت قرار گرفته استفاده می کرد، کد هوش مصنوعی نیز از اطلاعات RaycastHit برای تعیین اینکه آیا چیزی در مقابل دشمن است و اگر چنین است، چقدر دور است استفاده می کند.

- یکی از تفاوت های پخش پرتو برای تیراندازی و پخش پرتو برای هوش مصنوعی، شعاع پرتو است. برای تیراندازی، پایان خط پرتو بی نهایت است، اما برای هوش مصنوعی، پرتو دارای سطح مقطع بزرگ و گسترش آن محدود است.
- از نظر کد، این به معنای استفاده از متد SphereCast() به جای Raycast است. دلیل این تفاوت این است که گلوله ها کوچک هستند، در حالی که بررسی موانع در مقابل کاراکتر ما را ملزم می کند که عرض کاراکتر را در نظر بگیریم.





تا این جا ما می توانیم دشمن های داشته باشیم که با برخورد تیر از بین می روند
در ادامه می خواهیم این دشمن ها بتوانند در صحنه حرکت کنند
یک اسکریپت جدید به نام WanderingAI ایجاد کنید، آن را به شی مورد نظر (در کنار اسکریپت ReactiveTarget) وصل کنید
و کد زیر را در آن بنویسید.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class WanderingAI : MonoBehaviour {
    public float speed = 3.0f;
    public float obstacleRange = 5.0f;
    void Update() {
        transform.Translate(0, 0, speed * Time.deltaTime);
        Ray ray = new Ray(transform.position, transform.forward);
        RaycastHit hit;
        if (Physics.SphereCast(ray, 0.75f, out hit)) {
            if (hit.distance < obstacleRange) {
                float angle = Random.Range(-110, 110);
                transform.Rotate(0, angle, 0);
            }
        }
    }
}
```

مقادیر برای سرعت حرکت و فاصله ای
که در آن باید به موانع واکنش نشان داد

بدون توجه به چرخش، در هر
فریم بازیکن را به طور مداوم
به جلو حرکت می دهد.

پرتوی در مکان یکسان و در
جهتی مشابه با شخصیت قرار
میگیرد

همانطور که قبلاً توضیح داده شد، محاسبه raycasting با استفاده از روش Physics.Sphere-Cast() انجام می شود. این روش برای تعیین فاصله اطراف پرتو برای تشخیص تقاطع ها از یک پارامتر شعاع استفاده می کند، اما از هر نظر دقیقاً مشابه Physics.Raycast() است.



Tracking the character's state

بیا یاد تنظیمات کوچکی را در کد انجام دهیم تا زنده بودن شخصیت را ردیابی کنیم - یا به بیان دیگر (فنی تر)، می خواهیم وضعیت زنده شخصیت را ردیابی کنیم.

داشتن کد برای ردیابی و پاسخ متفاوت به وضعیت فعلی شی، یک الگوی برنامه نویسی رایج در بسیاری از زمینه های برنامه نویسی، نه فقط هوش مصنوعی است. پیاده سازی های پیچیده تر این رویکرد به عنوان ماشین های حالت (*state machines*) یا احتمالاً ماشین های حالت محدود (*finite-state machines*) نامیده می شوند.

ماشین حالت محدود (FSM) یک راهکار برنامه نویسی است که در آن وضعیت فعلی شی ردیابی می شود، انتقال های کاملاً مشخصی بین حالت ها وجود دارد و کد بر اساس وضعیت رفتار متفاوتی دارد.





اسکرپت بعدی یک مقدار بولین (isAlive) را به بالای اسکرپت اضافه می کند و کد نیاز به بررسی های شرطی های گاه به گاه آن مقدار دارد. با این بررسی ها، کد حرکت فقط زمانی اجرا می شود که دشمن زنده است.

...

```
private bool isAlive;
```

```
void Start() {
```

```
isAlive = true;
```

```
}
```

```
void Update() {
```

```
if (isAlive) {
```

```
transform.Translate(0, 0, speed * Time.deltaTime);
```

```
...
```

```
}
```

```
}
```

```
public void SetAlive(bool alive) {
```

```
isAlive = alive;
```

```
}
```

```
...
```

مقدار بولی برای ردیابی اینکه آیا دشمن زنده است یا خیر

و مقدار دهی اولیه به متغیر

تنها در صورتی حرکت کنید که شخصیت زنده باشد.

اسکرپت ReactiveTarget اکنون می تواند به اسکرپت WanderingAI بگوید که آیا دشمن زنده است یا خیر.

...

```
public void ReactToHit() {
```

```
WanderingAI behavior = GetComponent<WanderingAI>();
```

```
if (behavior != null) {
```

```
behavior.SetAlive(false);
```

```
}
```

```
StartCoroutine(Die());
```

```
}
```

...

بررسی کنید که آیا این شخصیت دارای اسکرپت WanderingAI است. ممکن است نباشد



Spawning enemy prefabs

در حال حاضر فقط یک دشمن در صحنه است و وقتی می میرد صحنه خالی است. بیا یاد کاری کنیم که بازی دشمنان را اتوماتیک تولید کند تا هر زمان که دشمن می میرد، دشمن جدیدی ظاهر شود. این کار در یونیتی با استفاده از طریق (prefab) به راحتی انجام می شود.

What is a prefab?

پیش ساخته ها (Prefabs) یک رویکرد انعطاف پذیر برای تعریف بصری اشیاء تعاملی هستند. به طور خلاصه، prefab یک شیء بازی کامل تر شده است (با کامپوننت های قبلاً متصل و تنظیم شده است) که در هیچ صحنه خاصی وجود ندارد، بلکه به عنوان یک دارایی وجود دارد که می تواند در هر صحنه کپی شود. اشیاء پیشرفته ایی که برای استفاده در بازی تولید می شوند و بیشتر برای اشیاء که زیاد تکرار می شوند کاربرد دارد

- یک کپی از یک پیش ساخته یک نمونه (*instance*) نامیده می شود،
- prefab به شی بازی موجود در خارج از هر صحنه اشاره دارد.
- Instance به یک کپی از شی که در یک صحنه قرار داده شده است اشاره دارد.





Creating the enemy prefab

برای ایجاد یک پیش ساخته، ابتدا یک شی در صحنه ایجاد کنید که تبدیل به پیش ساخته شود. از آنجا که شی دشمن ما به یک پیش ساخته تبدیل خواهد شد، ما قبلاً این مرحله را انجام داده ایم. اکنون تنها کاری که انجام می دهیم این است که شی را از نمای Hierarchy به پایین بکشیم و آن را در نمای پروژه رها کنیم. این به طور خودکار شی را به عنوان یک پیش ساخته ذخیره می کند

- در نمای سلسله مراتبی، نام شی اصلی به رنگ آبی در می آید تا نشان دهد که اکنون به یک پیش ساخته پیوند داده شده است.
- اکنون می توان شی دشمن را از صحنه حذف کرده و از prefab استفاده کنید
- اگر می خواهید پیش ساخته را بیشتر ویرایش کنید، کافی است روی پیش ساخته در نمای پروژه دوبار کلیک کنید تا باز شود و سپس روی فلش عقب در سمت چپ بالای نمای سلسله مراتبی کلیک کنید تا دوباره بسته شود.





Instantiating from an invisible SceneController

اگرچه خود پیش ساخته در صحنه وجود ندارد، یک شی باید در صحنه باشد تا کد تولید دشمن به آن متصل شود. ما یک شی بازی خالی ایجاد می کنیم و می توانیم اسکریپت را به آن متصل کنیم، اما شی در صحنه قابل مشاهده نخواهد بود.

- یک شی تهی تولید کرده و آن را در وسط صفحه در موقعیت (0,0,0) قرار دهید.
- یک اسکریپت با نام SceneController ایجاد کنید تا برای تولید اشیاء پیش ساخته استفاده کنیم.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class SceneController : MonoBehaviour {
    [SerializeField] GameObject enemyPrefab;
    private GameObject enemy;
    void Update() {
        if (enemy == null) {
            enemy = Instantiate(enemyPrefab) as GameObject;
            enemy.transform.position = new Vector3(0, 1, 0);
            float angle = Random.Range(0, 360);
            enemy.transform.Rotate(0, angle, 0);
        }
    }
}
```

یک متغیر سریال که به شی پیش ساخته دشمن که می خواهیم تولید شود وصل می کنیم

اگر دشمنی در صحنه نباشد
آنگاه یک دشمن جدید
ایجاد می شود

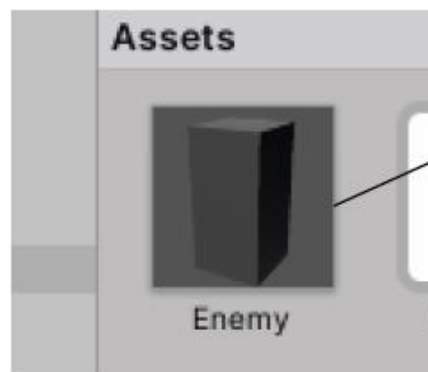
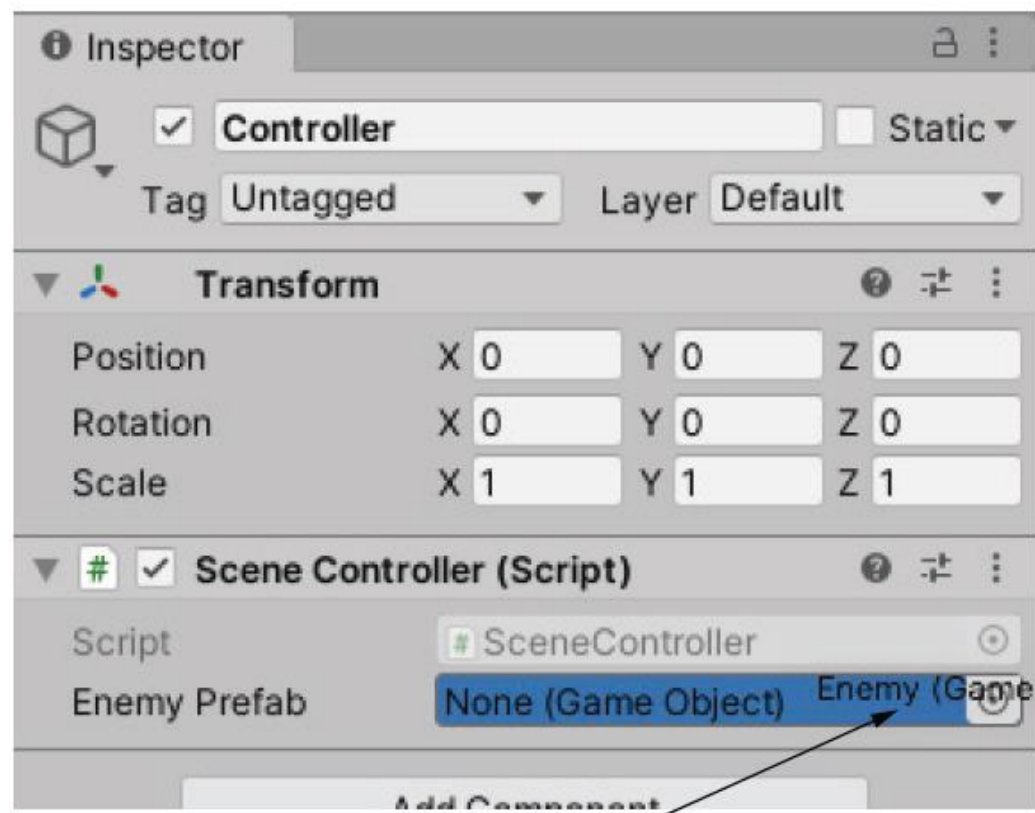
کپی کردن
شی پیش
ساخته



برای ارجاع به اشیا در ویرایشگر یونیتی، توصیه می‌کنم به جای تعریف متغیر `public`، آنها را با `SerializeField` تعریف کنید همانطور که در فصل 2 توضیح داده شد، متغیرهای `public` در پنجره کنترل ویرایشگر یونیتی `Inspector` نشان داده می‌شوند (به عبارت دیگر، آنها توسط `Unity` سریال می‌شوند)، بنابراین بیشتر آموزش‌ها و کد نمونه‌ای که مشاهده می‌کنید از متغیرهای عمومی برای همه مقادیر سریال استفاده می‌کنند. اما این متغیرها را می‌توان توسط اسکریپت‌های دیگر نیز تغییر داد (به هر حال این متغیرها عمومی هستند)، در حالی که ویژگی `Serialize-Field` به شما اجازه می‌دهد که متغیرها را خصوصی نگه دارید. اگر متغیری به صراحت عمومی نشده باشد، `C#` به صورت پیش‌فرض `private` در نظر می‌گیرد، و این در بیشتر موارد بهتر است زیرا می‌خواهید آن متغیر را در `Inspector` نمایش دهید اما نمی‌خواهید مقدار آن توسط اسکریپت‌های دیگر تغییر کند.

ممکن است لازم باشد در برخی از نسخه‌های قدیمی یونیتی حتماً متغیری از نوع `Serialize` را مقدار دهی اولیه کنید

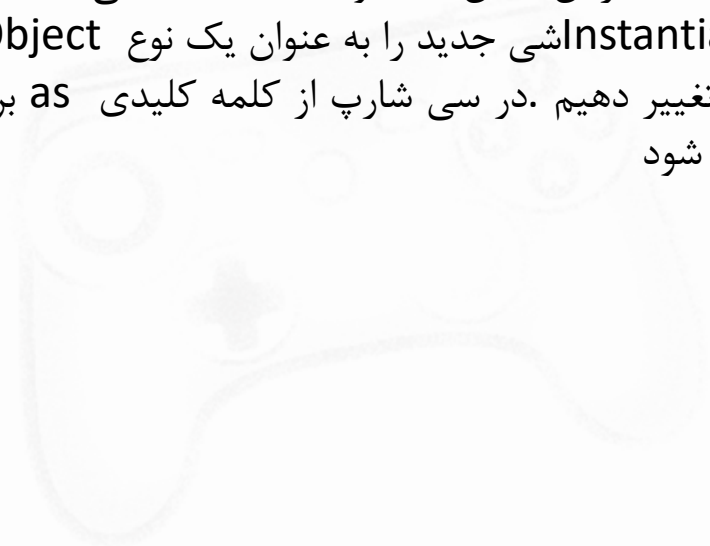




Drag the prefab up from
Project view to the slot
in the Inspector.



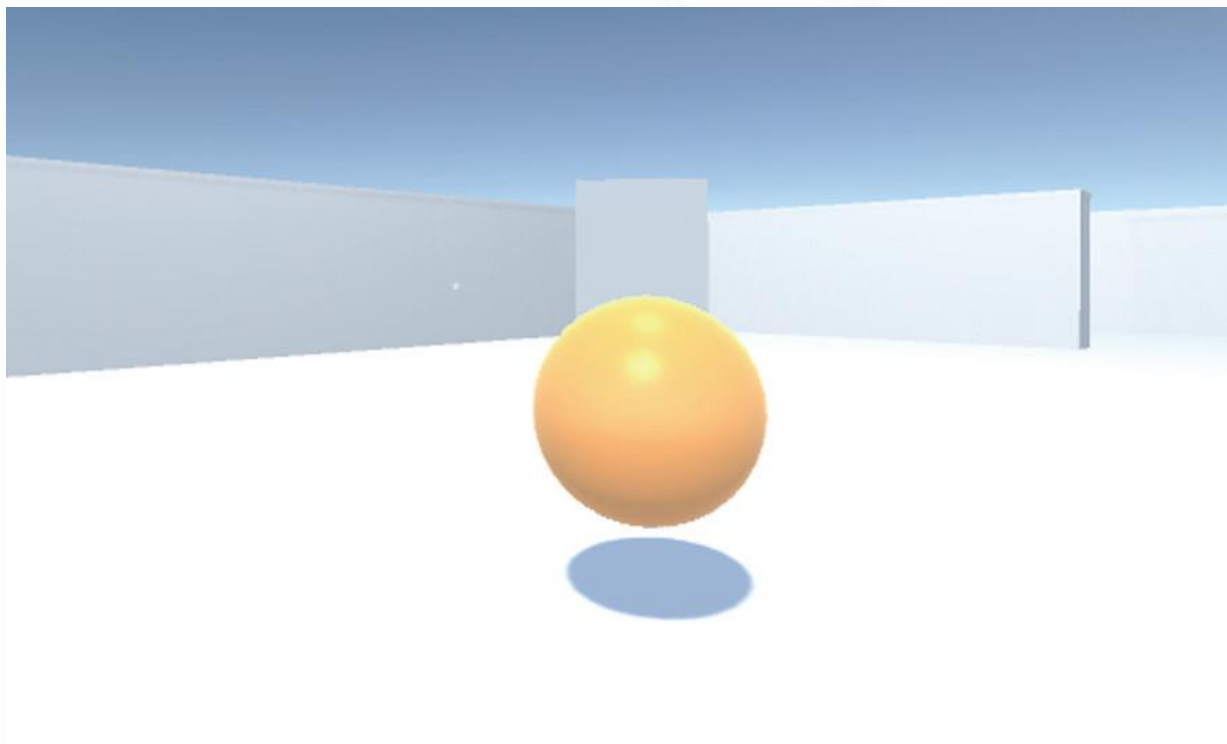
هسته اصلی این اسکریپت متد `Instantiate()` است، بنابراین به آن خط توجه کنید. هنگامی که ما نمونه ایی از `prefab` می سازیم، یک کپی در صحنه ایجاد می شود. به طور پیش فرض، `Instantiate()` شی جدید را به عنوان یک نوع `Object` عمومی برمی گرداند، اما `Object` مستقیماً بی فایده است و ما باید آن را به `GameObject` تغییر دهیم. در سی شارپ از کلمه کلیدی `as` برای `typecasting` تغییر نوع یک متغیر استفاده کنید تا از یک نوع شی به نوع دیگری تبدیل شود





Shooting by instantiating objects

بسیار خوب، بیایید کمی قابلیت دیگر به دشمنان اضافه کنیم. همانطور که با بازیکن انجام دادیم، ابتدا آنها را وادار به حرکت کردیم - حالا بیایید آنها را وادار به تیراندازی کنیم! همانطور که در هنگام معرفی `raycasting` اشاره کردم، این تنها یکی از رویکردهای اجرای تیراندازی بود. روش دیگر شامل نمونه سازی پیش ساخته ها است، بنابراین بیایید این رویکرد را برای وادار کردن دشمنان به تیراندازی در پیش بگیریم .



دشمن یک گوی آتشین به سمت بازیکن پرتاب می کند



همانطور که دشمن را بطور تصادفی تولید می کردیم می خواهیم گویی های آتشین را نیز تولید کنیم. با نمونه سازی یک prefab این کار را انجام می دهیم. همانطور که در بخش قبل توضیح داده شد، اولین مرحله هنگام ایجاد یک پیش ساخته، ایجاد یک شی در صحنه است که تبدیل به پیش ساخته می شود، بنابراین بیایید یک توپ آتشین بسازیم.

To start, choose GameObject > 3D Object > Sphere

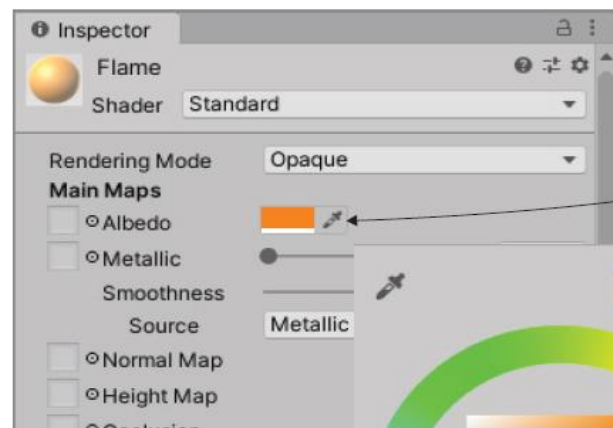
- یک گوی ایجاد کرده و اسم آن را Fireball بگذارید شما می توانید یک فایل اسکریپت به همین نام ایجاد کرده و آن را به جسم متصل کنید کد لازم را در مراحل بعدی به این فایل اضافه خواهیم کرد.
- برای آنکه گوی شبیه گوی آتشین به نظر برسد آن را رنگ آمیزی می کنیم
- خواص سطحی مانند رنگ با استفاده از material کنترل می شود.

material : بسته ای از اطلاعات است که ویژگی های سطح هر شی 3بعدی را که ماده به آن متصل است، تعریف می کند. این خواص سطحی می تواند شامل رنگ، براقی و حتی زبری ظریف باشد.

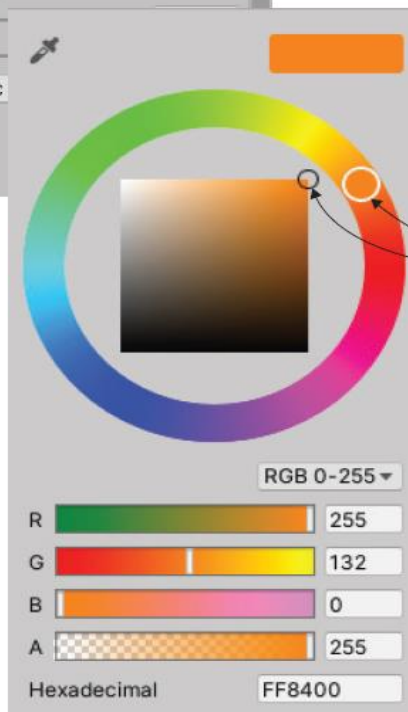
DEFINITION A *material* is a packet of information that defines the surface properties of any 3D object that the material is attached to. These surface properties can include color, shininess, and even subtle roughness.



Choose Assets > Create > Material.



Click the color swatch to bring up the color picker.



Adjust the hue on the outer ring and the value in the main area.

ما همچنین مواد را روشن می کنیم تا بیشتر شبیه آتش شود. مقدار Emission، یکی از ویژگی های دیگر در Inspector را تنظیم کنید. چک باکس به طور پیش فرض خاموش است، بنابراین آن را روشن کنید تا مواد روشن شود.



Shooting the projectile and colliding with a target

شلیک گلوله و برخورد با هدف

می خواهیم این توانایی را به دشمن اضافه کنیم که بتواند به سمت بازیکن گوی های آتشین پرتاب کند. از آنجا که کد برای شناسایی بازیکن به یک اسکریپت جدید نیاز دارد (درست مانند **ReactiveTarget** که توسط بازیکن برای شناسایی هدف مورد نیاز بود)، ابتدا یک اسکریپت جدید ایجاد کنید و نام آن را **PlayerCharacter** بگذارید. این اسکریپت را به شی پخش کننده در صحنه وصل کنید. حالا **WanderingAI** را باز کنید و کد را از این لیست اضافه کنید.

Add these two fields before any methods, just as in **SceneController**.

```
...  
[SerializeField] GameObject fireballPrefab;  
private GameObject fireball;  
...  
if (Physics.SphereCast(ray, 0.75f, out hit)) {  
    GameObject hitObject = hit.transform.gameObject;  
    if (hitObject.GetComponent<PlayerCharacter>()) {  
        if (fireball == null) {  
            fireball = Instantiate(fireballPrefab) as GameObject;  
            fireball.transform.position =  
            transform.TransformPoint(Vector3.forward * 1.5f);  
            fireball.transform.rotation = transform.rotation;  
        }  
    }  
}
```

Player is detected in the same way as the target object in **RayShooter**.

Instantiate() method here is just as it was in **SceneController**.

Place the fireball in front of the enemy and point in the same direction.

Same null Game-Object logic as **SceneController**



Shooting the projectile and colliding with a target

Add these two fields before any methods, just as in SceneController.

```
...
[SerializeField] GameObject fireballPrefab;
private GameObject fireball;
...
if (Physics.SphereCast(ray, 0.75f, out hit)) {
    GameObject hitObject = hit.transform.gameObject;
    if (hitObject.GetComponent<PlayerCharacter>()) {
        if (fireball == null) {
            fireball = Instantiate(fireballPrefab) as GameObject;
            fireball.transform.position =
                transform.TransformPoint(Vector3.forward * 1.5f);
            fireball.transform.rotation = transform.rotation;
        }
    }
    else if (hit.distance < obstacleRange) {
        float angle = Random.Range(-110, 110);
        transform.Rotate(0, angle, 0);
    }
}
...
```

Same null
Game-Object
logic as
SceneController

Player is detected in the
same way as the target
object in RayShooter.

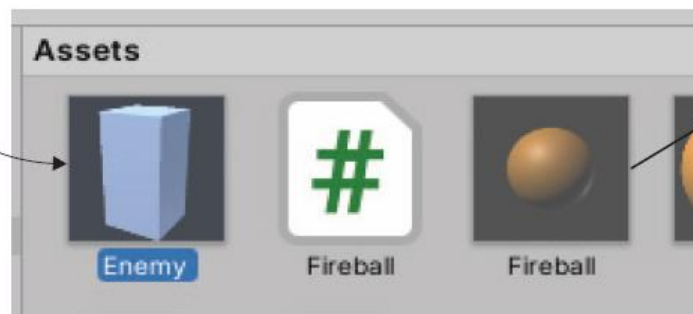
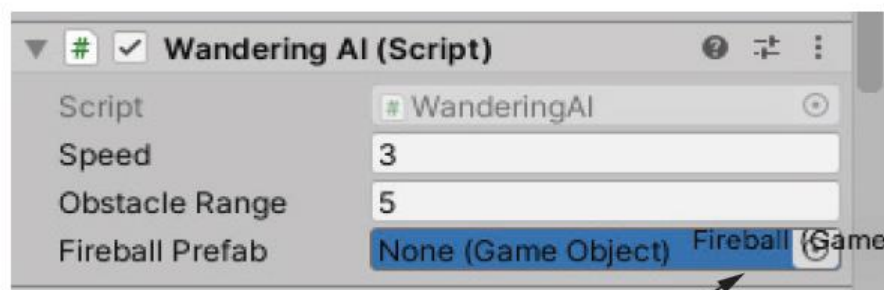
Instantiate()
method here is just
as it was in
SceneController.

Place the fireball in
front of the enemy
and point in the same
direction.



- هوش مصنوعی دشمن با raycast بررسی می کند اگر شیء که در برابر آن است بازیکن باشد گوی آتشین را به سمت بازیکن پرتاب می کند
 - گوی آتشین یک پیش ساخته است که نمونه آن در متغیر سریال قرار می گیرد
 - برای مشخص کردن بازیکن از سایر اشیاء از PlayerCharacter استفاده می کنیم. این کد را به بازیکن متصل می نماییم.
 - به یاد داشته باشید که برای مشخص کردن اهداف که بازیکن می تواند به آنها شلیک کند از ReactiveTarget استفاده کردیم.
-
- به کد هوش مصنوعی دشمن یک متغیر جدید اضافه شده است که باید آن را با پیش ساخته گوی آتشین مقدار دهی کنیم
 - پیش ساخته دشمن در پنجره پروژه در قسمت پایین انتخاب کرده و روی آن دابل کلیک کنید حال می توانید گوی پیش ساخته را کشیده و مطابق تصویر زیر متغیر جدید را مقدار دهی کنید

Select the Enemy prefab to show its components in the Inspector.



Drag the Fireball prefab from Project view to the slot in the Inspector.



گوی آتشین راه شده توسط کارکتر دشمن باید رو به جلو شروع به حرکت کند و همچنین در صورت برخورد با اجسام عکس العمل مناسب را از خود نشان دهد. کد زیر به این منظور نوشته شده است.
یک فایل اسکریپت به نام Fireball تعریف کرده و کد زیر را به آن اضافه می کنیم.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class Fireball : MonoBehaviour {
    public float speed = 10.0f;
    public int damage = 1;
    void Update() {
        transform.Translate(0, 0, speed * Time.deltaTime);
    }
    void OnTriggerEnter(Collider other) {
        PlayerCharacter player = other.GetComponent<PlayerCharacter>();
        if (player != null) {
            Debug.Log("Player hit");
        }
        Destroy(this.gameObject);
    }
}
```

Move forward in the
direction it faces.

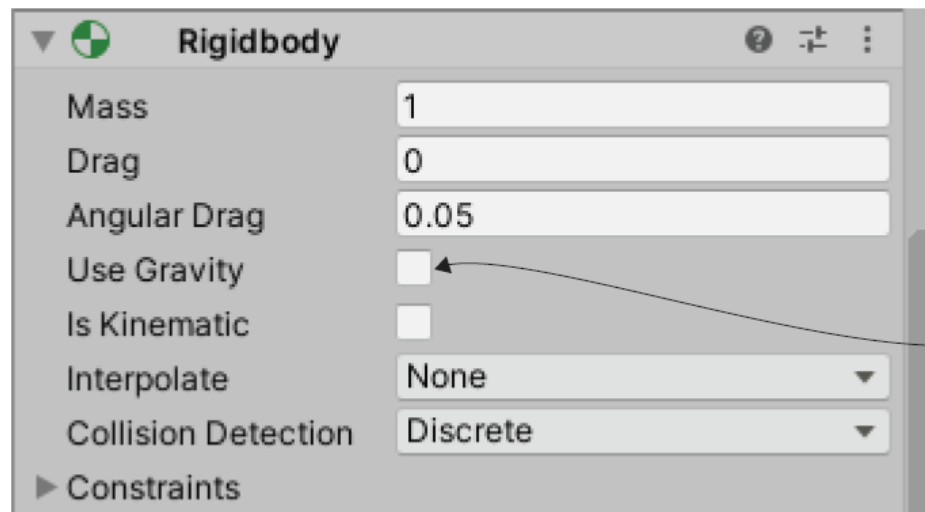
Called when another
object collides with
this trigger

Check if the other object
is a PlayerCharacter.



- تابع `OnTriggerEnter` بطور خودکار وقتی جسم به یک جسم دیگر برخورد می کند فراخوانی می شود و می توان به کمک آن برخوردها را شناسایی کرد
- البته هنوز کد کامل نشده است و چند تنظیم دیگر باید در اجزای موجود در شی `Fireball` انجام شود.
- اولین تغییر این است که برخورد دهنده را به یک `Trigger` تبدیل می کند. برای تنظیم آن، به `Inspector` بروید و روی کادر `Is Trigger` در مولفه `Sphere Collider` کلیک کنید.
- گوی آتشین همچنین به یک `Rigidbody` نیاز دارد، تا موتور فیزیک یونیتی بتواند بخوبی شبیه سازی فیزیکی و برخورد را انجام دهد با دادن یک جزء `Rigidbody` به توپ آتشین، مطمئن می شوید که سیستم فیزیک قادر به ثبت محرک های برخورد برای یک شی است.
- در کامپوننتی که اضافه شده است، گزینه `Use Gravity` را بردارید

وقتی کامپوننت `collider` یک جسم به عنوان یک `Trigger` تنظیم شده است، جسم به برخورد و مماس شدن به ی اشیاء دیگر واکنش نشان می دهد، اما مشکلی در سایر شبیه سازی های فیزیکی ایجاد نمی کند.



Uncheck this value.



Damaging the player

قبلاً یک اسکریپت `PlayerCharacter` ایجاد کردید اما آن را خالی گذاشتید. اکنون کدی را می نویسید تا بازیکن به ضربه خوردن (تیر خوردن) واکنش نشان دهد.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class PlayerCharacter : MonoBehaviour {
    private int health;
    void Start() {
        health = 5;
    }
    public void Hurt(int damage) {
        health -= damage;
        Debug.Log($"Health: {health}");
    }
}
```

Initialize the health value.

Decrement the player's health.

Construct the message by using string interpolation.



این کد یک فیلد برای سلامت (جان) بازیکن تعریف می کند و سلامتی را به طور متناسب کاهش می دهد. در فصل های بعدی، برای نمایش اطلاعات روی صفحه نمایش از این اطلاعات استفاده خواهیم کرد، اما در حال حاضر، فقط با استفاده از پیام های اشکال زدایی می توانیم اطلاعات مربوط به سلامت بازیکن را نمایش دهیم.

اکنون باید به اسکریپت Fireball برگردید تا متد `Hurt()` بازیکن را فراخوانی کنید. شما می توانید این تابع را به جای دستور `Debug.Log("Player hit")` فراخوانی کنید تا به بازیکن بگویید که ضربه خورده است.

